

Independent Ladisa classification

Instructions for the second rater

This exercise asks whether a software provenance system would produce a detectable signal for each attack position in a published software supply chain taxonomy. It does not ask whether the system prevents the attack.

Complete the supplied Excel workbook independently. Do not consult the candidate's thesis results, evaluation tables, completed classifications, or reported coverage percentage before returning the workbook.

What you received

- This PDF guide.
- An Excel workbook containing 114 classifiable positions from the pinned taxonomy.
- A CSV copy of the blank classification sheet, for use only if Excel-compatible software is unavailable.

What you do not need

- No GitHub account or repository access.
- No blockchain wallet, private key, funds, software installation, or command-line work.
- No access to the candidate's private data.
- No prior knowledge of Layer-2 networks.

Independence

Please stop and tell the candidate if you have already seen the completed classification or its percentage. You may use the Ladisa paper to clarify an attack term, but record that use on the Rater information sheet. Do not use the candidate's thesis or public evaluation write-up as a source.

System profile

The system records a commitment to a Git release state on a public Ethereum Layer-2 network. When a release tag is created, CI resolves the tag to a commit and submits the root Git tree identifier. The transaction may also bind the SHA-256 digest of a generated CycloneDX SBOM. A project identifier and reference identify the record.

What later verification can compare

- A current Git reference can be resolved and its tracked root tree compared with the earlier on-chain tree identifier.
- A source release archive can be reconstructed into a tree-shaped view and compared with the anchored tree.
- Tracked file contents, file modes, subtrees, and submodule object identifiers contribute to the Git tree.
- A supplied SBOM file can be hashed and compared with the SBOM digest stored beside the tree commitment.
- Scheduled re-verification can repeat the Git-reference comparison without submitting a new transaction.

Boundaries to apply

- The system is detective. A detectable label means a later check can produce a signal. It does not mean the attack is blocked.
- The system does not establish that source code is benign. Malicious code already present when the first honest-looking anchor is written can be recorded faithfully.
- An allowlisted CI credential controls submission. If that credential or the submitting CI environment is compromised before the first false anchor, the blockchain accepts the signed submission.
- Compiled binaries are not checked for correspondence with source. A binary-only change therefore has no source-tree mismatch.
- Schema version 1 permits declared generated-extra paths in a source archive. Those paths are removed before tree comparison, and their contents are not authenticated.
- An undeclared extra path or a changed tracked file changes the reconstructed tree and can produce a mismatch.
- A different package chosen through name confusion does not automatically contradict the genuine project's anchor. Verification is bound to a project identifier rather than a package name.
- The public record can show that a newer anchored release exists, but it cannot force a distributor to deliver that release.
- For reference tampering, assess the state after an honest anchor exists. A false first anchor made with the authorised anchoring credential falls under the first-anchor boundary above.

Classification labels

Label	When to use it
detectable	A later check can produce a cryptographically checkable mismatch or other explicit signal from the anchored record.
detectable-conditional	The signal depends on a distinction the taxonomy position does not carry, such as whether build tampering changes a source archive, a compiled binary, or a declared generated-extra path.
partial	The supplied object can remain genuine and match its commitment, but the public record can reveal related evidence, such as the existence of a newer anchored release.
out-of-scope	The system has no relevant counterpart to compare, faithfully records the bad state, relies on the compromised anchoring process, or does not compare the changed object.

How to classify each row

Read the attack name together with its full taxonomy path. The same AV identifier can occur beneath different parents and may receive a different answer in each context.

- Ask what state the attack changes: tracked source, source archive, Git reference, package selection, compiled binary, or the anchoring process itself.
- Ask whether an honest anchor exists before the changed state.
- Ask what a later verifier can compare with that anchor.
- Choose one label. If relevant facts vary inside the taxonomy position, use detectable-conditional rather than selecting the favourable case.
- Write one or two sentences explaining the signal or the absence of one.
- Give a confidence rating if useful and record any unresolved ambiguity.

Practice example (not a scored taxonomy row)

Suppose an honest release tree is anchored. A distributor later changes the contents of a tracked README file in the source archive. The changed archive reconstructs to a different tree, so a verifier can observe a mismatch. The appropriate label is **detectable**. This does not mean the system prevented distribution of the changed archive.

Completing and returning the workbook

- Complete the Rater information sheet.
- Enter one classification for every row in the Classification sheet.
- Give a short reason for every classification.
- Do not alter instance IDs, AV IDs, attack names, or taxonomy paths.
- Do not calculate agreement or a coverage percentage.
- Save the workbook under a new filename that includes your rater ID.
- Return the workbook to the candidate. Keep your first completed version unchanged if disagreements are discussed later.

Taxonomy provenance

The workbook uses *taxonomy.json* from SAP's Risk Explorer for Software Supply Chains, pinned at commit 79e2a3946a24cde1755809a50e54ee15e3af893e. The exact source URL and counts are recorded in the workbook. The source tree contains repeated vector identifiers in different parent contexts; the workbook retains each position.

Questions during classification

You may ask for clarification of a system fact or an unfamiliar term. The candidate should answer without showing or stating the original classification. Record any clarification that materially affected a decision in the workbook's question column.